

PRÁCTICA 2

Elena Abreu Fernández y Tsan-Yu Wu



**UNIVERSIDAD
DE GRANADA**

1. TokenFilters

Usaremos el siguiente texto en español para probar el efecto que tienen los siguientes tokenFilters:

“Muchos años después, frente al pelotón de fusilamiento, el coronel Aureliano Buendía había de recordar aquella tarde remota en que su padre lo llevó a conocer el hielo.”

Como base usamos un StandardTokenizer, pues divide el texto en tokens coherentes e intuitivos (palabra a palabra) y nos ayudará a visualizar mejor los diferentes efectos de los filtros que a continuación aplicaremos :

- **LowerCaseFilter** : Así, observamos además que LowerCaseFilter modifica estos tokens de manera que todos aparezcan en minúsculas. Este primer filtro es básico para poder buscar coincidencias de manera eficiente, pues el uso de mayúsculas no suele modificar el significado de las palabras, por lo que no tiene sentido discriminar las palabras por ello.

Tokens generados:

- muchos
- años
- después
- frente
- al ...

- **StopFilter** : Este filtro se encarga de eliminar artículos, preposiciones, conjunciones, etc, lo que se conoce como “palabras vacías”, que no aportan sentido semántico al texto pero son muy frecuentes. Por ello, es un filtro que facilita mucho las búsquedas, al reducir considerablemente el tamaño del índice. Sin embargo, hay que tener en cuenta que a veces estas “palabras vacías” pueden ser claves según qué contexto, véase el título de una película.

Tokens generados:

- años
- después
- frente
- pelotón...

- **SnowballFilter**: este filtro se encarga de hacer stemming, es decir, eliminar las terminaciones de los token para dejar solo la raíz. En este caso aplicaremos claramente SpanishStemming, que eliminará terminaciones como -a, -o, -ía, -s, ..., además de las tildes. Con ello conseguiremos agrupar variantes de una misma palabra sin discriminar por género o número, aunque debemos observar detenidamente el método de stemming que usemos, pues algunos pueden ser muy agresivos y hacer agrupaciones erróneas.

Tokens generados:

- much
- años
- despues
- frent
- al
- peloton

- de
- fusil...
- **ShingleFilter** : genera tokens formados por una y dos palabras, esto ayuda a entender mejor el contexto de la búsqueda, sin embargo, aumenta el número de tokens con el que tratamos.

Tokens generados:

- muchos
- muchos años
- años
- años después
- después ...
- **EdgeNGramCommonFilter**: en nuestro caso le pasamos como parámetro el número 3, por lo que tokeniza usando los 3 primeros caracteres de cada palabra. Es muy útil para recomendaciones por prefijo, aunque genera tokens sin un sentido semántico, lo que puede complicar otro tipo de búsquedas.

Tokens generados:

- muc
- año
- des
- fre
- pel...
- **NGramTokenFilter**: indicamos en los parámetros que se formen tokens de entre 2 y 4 caracteres. Se diferencia del anterior filtro en que tokeniza usando todos los caracteres de las palabras, no solo los iniciales. Este filtro aumenta con creces el número de tokens con el que trabajaremos, pero puede ser de gran ayuda cuando el usuario comete algún error ortográfico en su búsqueda.

Tokens generados:

- mu
- muc
- much
- uc
- uch
- ucho
- ch
- cho
- chos
- ho
- hos
- os
- ...
- **CommonGramsFilter**: crea tokens de palabras combinadas, uniendolas con “palabras vacías” en lugar de eliminarlas como hace StopFilter. De nuevo, es un filtro que nos permite conocer más del contexto de la búsqueda y, al no eliminar las palabras vacías, evita posibles pérdidas, aunque esto suponga un aumento del número de tokens.

Tokens generados:

- muchos

- muchos_años
- años
- después
- frente
- frente_al
- al
- al_pelotón ...

- **SynonymFilter:** Para probar correctamente el efecto de este filtro hemos creado un pequeño diccionario de sinónimos específicos para el texto que estamos trabajando:

*“años,tiempo
frente,delante
pelotón,grupo
padre,papá,progenitor
hielo,frío”*

Observamos que a los tokens generados por StandardTokenizer se le añaden los sinónimos correspondientes, lo que nos permite enriquecer semánticamente el índice y mejorar la recuperación de información :

Tokens generados:

- muchos
- tiempo
- años
- después
- delante
- frente
- al
- grupo
- pelotón...

2. Analizador para X

A continuación, se nos pide crear un analizador para la red social X. Para ello, hemos optado por usar el tokenizador Whitespace, ya que, al no tratarse de una herramienta para analizar documentos formales sino una red social en la que se usa un lenguaje informal, emojis, siglas, etc. hemos pensado que sería más conveniente diferenciar tokens meramente por los espacios en blanco. Esto nos ha permitido además solucionar la problemática de que se incluyan los hashtags y las menciones como token válidos, preservando así elementos característicos de la comunicación en esta red social que habrían sido ignorados en caso de usar, por ejemplo, StandardTokenizer como se hizo en el apartado anterior.

Además, a este tokenizer le hemos añadido filtros como LowerCaseFilter para no tener tokens duplicados con mismo valor semántico, StopFilter para reducir el número de tokens y reducirlos únicamente a palabras no vacías, y SnowBallFilter con la posibilidad de aplicar stemming en tres idiomas distintos,. Para adaptar este analizador a X hemos creado un mapa de sinónimos con los siguientes emojis :

:)=feliz,contento,alegre
:=(triste,desanimado
T_T=llorar,llorando,deprimido

XD=reir,risa,divertido
>:(=enfadado,molesto
:D=euforico,entusiasmado
.3.=beso
<3=corazon,amor
</3=desamor,decepcion
:O=sorpresa

Gracias a esto el analizador es capaz de comprender el significado de estos símbolos, mejorando la calidad de los resultados de las tareas de búsqueda.

3. Sugerencias de consultas

Por último, vamos a realizar un análisis empírico de qué analizadores funcionan de manera más eficiente a la hora de hacer sugerencias al usuario.

Para comparar los resultados de los analizadores y los suggesters (PrefixSuggester, FuzzyPrefixSuggester, InfixSuggester), definimos primero una lista de palabras que introducimos en la búsqueda. La lista sea: ["at", "los", "coffees", "parking", "city"]. La lista es diseñada para ver si cada analizador se comporta de manera diferente si se introducen palabras vacías ("at" en inglés y "los" en español), el plural, forma diferente de un verbo (parking). Probamos en el orden: WhiteSpaceAnalyzer, SimpleAnalyzer, StandardAnalyzer, EnglishAnalyzer, SpanishAnalyzer, y ShingleAnalyzer. Compararemos siempre con el resultado del analizador anterior excepto el primero.

Comenzamos probando el WhitespaceAnalyzer, los resultados de cada suggester si introducimos las palabras de la lista definida una a una:

- PrefixSuggester: ninguna recomendación.
- FuzzyPrefixSuggester: ninguna recomendación.
- InfixSuggester: excepto "coffees" y "city", las recomendaciones son de "amenities" que contienen la palabra que introducemos como "... at extra cost", "... lossless ...", "Free street parking".

Entonces al usar el analizador la primera problemática a la que nos enfrentamos es que esta herramienta discrimina el uso de mayúsculas y minúsculas y no hace stemming, por lo que no es recomendable usarlo en este caso, pues es muy común que los usuarios no hagan uso de las mayúsculas lo que resultaría en recomendaciones vacías al buscar cualquier barrio de la ciudad.

Continuamos con el SimpleAnalyzer:

- PrefixSuggester: excepto "coffees" y "parking", las recomendaciones son por ejemplo, "Atwood", "Los Cerritos" y "City skyline view" (de "amenities").
- FuzzyPrefixSuggester: casi los mismos resultados que PrefixSuggester, pero con la consulta "los" se recibe las recomendaciones por ejemplo "Las Lomas I" y "Lock on bedroom door", sin ninguno barrio cuyo nombre contiene "Los", y con "coffees", la respuesta contiene "Coffee"(de "amenities). Con "city" "Citrus Grove" está también en las recomendaciones.

- InfixSuggester: Con “at”, “parking”, “city”, las recomendaciones son de “amenities” que contienen la palabra que introducimos. Con “los”, las recomendaciones son barrios de la ciudad como “North Los Altos”, “Los Altos”. (nada de “amenities”). Ninguna recomendación para “coffees”.

Luego probamos el StandardAnalyzer: los resultados son iguales que SimpleAnalyzer en nuestro caso.

Nos parece raro que los analizadores SimpleAnalyzer, StandardAnalyzer tengan los mismos resultados aunque StandardAnalyzer debe filtrar las palabras vacías como “at”, pero no era así.

Seguimos probando el EnglishAnalyzer: ningún de los 3 suggesters tienen recomendaciones con “at”, porque es una palabra vacía en inglés. El resto:

- PrefixSuggester: las recomendaciones de “city” siguen las mismas. Con “los” hay "Lock on bedroom door", "Loyola Heights" en el resultado también, pero en total las recomendaciones parecen menos precisas. Con “coffees” se consigue las recomendaciones de “amenities” que contienen "Coffee". Las recomendaciones de “parking” son los barrios (“Park West”, “Parkside”, etc.)
- FuzzyPrefixSuggester: las recomendaciones de “city” y “coffee” siguen las mismas. Con “los”, esta vez hay no solo las de “amenities” ("Lock on bedroom door", "Locally sourced conditioner") sino los barrios (p.e. Los Feliz) también. Con “parking”, las recomendaciones son de PrefixSuggester y algunas más (p.e. Paramount)
- InfixSuggester: excepto “los”. Los demás tienen las mismas recomendaciones. Con “los”, las recomendaciones no son barrios, sino los elementos de “amenities” que contiene la palabra “closet”.

El mayor problema que encontramos al usar el EnglishAnalyzer es que si el usuario introduce una palabra vacía, no recibirá ninguna sugerencia. Como EnglishAnalyzer usa lemmatization y stemming, la precisión y la relevancia se reduce. Aunque esto no sea siempre malo (en algunos casos parece que hay más recomendaciones), hay que tenerlo en cuenta.

Continuámos con el SpanishAnalyzer: como “los” es una palabra vacía en español, ninguna sugerencia se recibe. El resto:

- PrefixSuggester: las recomendaciones son casi iguales que las con EnglishAnalyzer, pero como “parking” no es una forma de verbo en español, no se recibe ninguna sugerencia y con “at”, las recomendaciones son otra vez los barrios (p.e. Atwood)
- FuzzyPrefixSuggester: las recomendaciones de “city” y “coffee” siguen las mismas. Y con “at” y “parking” los resultados son iguales que PrefixSuggester.
- InfixSuggester: los resultados son casi como con el uso de EnglishAnalyzer, hasta “los”. Además con “at” ahora se recibe sugerencias de “amenities” que contienen esta palabra.

También encontramos ciertos problemas al usar el SpanishAnalyzer pues, aunque el usuario que use el buscador puede ser hispano hablante, hay que tener en cuenta que los nombres de los barrios indexados están en inglés. Así, si por ejemplo introducimos “a” o “al”,

mientras que con el resto de analizadores ya comenzamos a recibir sugerencias, debido al stemming español, no recibiríamos ninguna.

Al final probamos el ShingleAnalyzer:

El primer problema que encontramos es que para construir un Suggester, mucho más memoria es necesaria. No podemos ejecutar el programa con éxito. Por otro lado observamos que ShingleAnalyzer puede llegar a ser muy útil al tomar dos palabras como un único token lo que facilita sugerir barrios como “Santa Mónica”, “West Hollywood” o características de los apartamentos como “air fryer” o “coffee maker”. Sin embargo, al aumentar tanto el número de tokens con el que se trabaja la velocidad del programa se ve muy afectada, lo que perjudica la experiencia de usuario, que busca no solo ser eficaz sino también rápida.

Tras comparar el comportamiento de los distintos analizadores y suggesters, podemos afirmar que no existe un único analizador óptimo en todos los casos, sino que la elección depende del tipo de datos y de la experiencia de usuario que se quiera conseguir. Teniendo en cuenta todos los pros y contras que hemos listado anteriormente y valorando tanto la eficacia de las sugerencias como la rapidez de respuesta y la naturalidad de uso para mejorar la experiencia de usuario, la combinación que ofrece el mejor equilibrio es StandardAnalyzer y PrefixSuggester.

4. Trabajo en grupo

El primer y segundo apartado han sido realizados por Elena Abreu Fernández.

El tercer apartado ha sido realizado por Tsan-Yu.